# COMPUTER ARCHITECTURE FOR CYBERSECURITY

MASTER 7-MODULE SYLLABUS, HARDWARE EXPLOIT MAPPINGS & COMPETENCY CHECKLIST

**Standard:** System V AMD64 ABI & Linux/Windows Internals | **Track:** Foundations Tier 1 | **Author:** dvsh03

## MODULE 01: Silicon Foundations & 5-Stage Execution Cycle

- **Von Neumann vs Harvard:** Unified data/code memory (exploits rely on this) vs split physical caches.
- **CPU Subsystems:** ALU (integer/logic math), Control Unit (microcode, clock), internal SRAM registers.
- **System Buses:** 64-bit Address bus (canonical 48-bit address space), Data bus, Control bus (IRQs).
- **5-Stage Pipeline:** Fetch (RIP read), Decode (opcode parse), Execute (ALU), Memory access, Write-back.
- **Branch Prediction:** Speculative branch execution hardware (fundamental root of Spectre attacks).

**Hands-on Labs:** `Lab 1.1: Python architecture & hardware probe | Lab 1.2: CPU rdtsc cycle measurement`

## MODULE 02: x86_64 Register Map, Calling Conventions & ARM64

- **16 General Purpose Registers:** RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP, R8 through R15.
- **Sub-Register Slicing:** 64b (RAX) -> 32b (EAX) -> 16b (AX) -> 8b Low (AL) & 8b High (AH).
- **System V AMD64 ABI:** Args in RDI, RSI, RDX, RCX, R8, R9. Return in RAX. Callee vs Caller saved.
- **Windows x64 Fastcall:** Args in RCX, RDX, R8, R9 with mandatory 32-byte Shadow Space on stack.
- **ARM64 Architecture:** RISC fixed 32-bit instructions, X0-X30 registers, Link Register (LR / X30).
- **Exploit Insight:** Null-byte elimination via XOR tricks (xor eax, eax vs mov rax, 0) for compact shellcode.

**Hands-on Labs:** `Lab 2.1: GDB register tracing & disassembly | Lab 2.2: Pure NASM assembly argument passing`

## MODULE 03: Process Memory Layout, Endianness & Stack Frames

- **Virtual Memory Segments:** .text (RX), .rodata (R), .data (RW), .bss (RW), Heap (up), Stack (down).
- **Endianness Mechanics:** Little Endian (x86_64 LSB at low addr) vs Big Endian (Network byte order).
- **Exploit Reversal:** Target address 0x7fffffffe420 injected as \x20\xe4\xff\xff\xff\x7f\x00\x00.
- **Stack Frame Lifecycle:** 16-byte alignment, call push RIP, Prologue (push rbp; mov rbp, rsp), Epilogue.
- **Buffer Overflow Roots:** Memory writes grow upwards towards high memory, trampling Saved RBP and Saved RIP.

**Hands-on Labs:** `Lab 3.1: Python endianness memory simulator | Lab 3.2: GDB stack frame dissection (x/20gx $rsp)`

## MODULE 04: Memory Hierarchy, Caches & Hardware Side-Channels

- **Hardware Pyramid:** Registers (0.5ns) -> L1 Cache (1ns) -> L2 Cache -> L3 LLC -> RAM -> SSD/Disk.
- **Cache Lines:** 64-byte line blocks, Cache hits vs Cache misses, Temporal & Spatial Locality.
- **Speculative Execution:** Out-of-Order Execution (OoOE) leaves microarchitectural state in cache lines.
- **Meltdown (CVE-2017-5754):** Rogue data cache load bypassing user/kernel boundary before privilege trap.
- **Spectre (CVE-2017-5753/5715):** Tricking branch prediction to leak secret memory via Flush+Reload timing.
- **Rowhammer:** Rapid DRAM row activation causing electromagnetic disturbance to flip adjacent memory bits.

**Hands-on Labs:** `Lab 4.1: C cache access timing probe via rdtscp | Lab 4.2: Flush+Reload cache side-channel PoC`

# COMPUTER ARCHITECTURE: ADVANCED MODULES & EXIT BENCHMARKS

MEMORY MANAGEMENT, PRIVILEGE ISOLATION, HARDWARE DEFENSES & AUDIT CRITERIA

## MODULE 05: MMU, Virtual Memory, Multi-Level Paging & Page Tables

- **Memory Management Unit (MMU):** Virtual-to-physical address translation; complete inter-process isolation.
- **4-Level Paging (x86_64):** 48-bit address decomposed into PML4 -> PDPT -> PD -> PT -> 4KB Physical Page Frame.
- **CR3 Control Register:** Stores physical base address of the PML4 table for active process.
- **PTE Bit Flags:** Present (P), Read/Write (R/W), User/Supervisor (U/S), and No-Execute (NX / XD Bit 63).
- **Translation Lookaside Buffer (TLB):** Fast translation cache, TLB misses, and invlpg invalidation.
- **Page Faults:** Soft page fault (zero-fill on demand), Hard page fault (disk swap), SIGSEGV access violation.

**Hands-on Labs:** `Lab 5.1: Inspect /proc/[pid]/maps & smaps | Lab 5.2: C mprotect() dynamic PTE permission violation`

## MODULE 06: Hardware Privilege Rings, System Calls & Kernel Transitions

- **CPU Protection Rings:** Ring 0 (Kernel/Supervisor Mode) vs Ring 3 (User Applications) vs Ring -1 (Hypervisor).
- **Transition Mechanics:** Software interrupts (int 0x80 / int 0x2e via IDT) vs modern fast syscall (syscall / sysret).
- **The IA32_LSTAR MSR:** Model-Specific Register storing the kernel entry function pointer (entry_SYSCALL_64).
- **Hardware Syscall Steps:** CPU saves RIP->RCX, RFLAGS->R11, sets CPL to Ring 0, jumps to IA32_LSTAR.
- **Context Switching:** Kernel stack setup (swapgs), saving register context to pt_regs, sys_call_table dispatch.

**Hands-on Labs:** `Lab 6.1: System call tracing via strace -c -e trace=all | Lab 6.2: Raw assembly standalone syscall binary`

## MODULE 07: Hardware-Enforced Security Mitigations & CPU Defenses

- **Hardware NX/DEP:** Bit 63 of Page Table Entry blocks code execution on stack/heap (W^X). Bypassed by ROP.
- **ASLR & PIE:** Randomizing memory base addresses on execution; entropy limits in 32b vs 64b architectures.
- **Stack Canaries:** Compiler inserted 8-byte canary (FS:0x28) verified before ret (__stack_chk_fail).
- **Kernel SMEP (CR4 Bit 20):** Supervisor Mode Execution Prevention blocks kernel from executing user pages.
- **Kernel SMAP (CR4 Bit 21):** Supervisor Mode Access Prevention blocks kernel from reading/writing user pages.
- **Intel CET:** Hardware Shadow Stacks (immune to data stack ROP overwrites) and Indirect Branch Tracking (ENDBR64).

**Hands-on Labs:** `Lab 7.1: Binary security posture audit via checksec | Lab 7.2: Stack canary disassembly analysis in GDB`

## FINAL PHASE 1 ARCHITECTURE COMPETENCY BENCHMARKS (EXIT GATES)

1. **GDB Autonomy:** Disassemble any x86_64 ELF binary, inspect registers, evaluate stack frames, and trace call sequences.
2. **Calling Convention Precision:** Predict and verify all 6 System V argument registers and return values in RAX.
3. **Endianness Mastery:** Convert raw memory pointers into reversed Little-Endian shellcode payloads without external utilities.
4. **4-Level Page Table Translation:** Articulate the full virtual-to-physical translation process from CR3 to physical offset.
5. **Kernel Syscall Lifecycle:** Trace syscall invocation from Ring 3 through IA32_LSTAR MSR to Ring 0 execution and sysretq.